

pdfexplore Notes

Mission

- Keep `pdfexplore` read-only for source PDFs; persist only sidecar metadata.
- Maintain mdexplore-style interaction contracts where behavior maps cleanly to PDF workflows.
- Keep GUI responsive under medium/large trees by prioritizing user interaction over background cache work.

Product Narrative

`pdfexplore` exists to make high-volume PDF triage feel as fluid as markdown exploration while preserving strict document safety. The core idea is simple:

- source PDFs are immutable,
- user intent is captured as sidecar metadata,
- expensive work is pushed off the UI thread,
- the interface always favors interaction smoothness over background throughput.

The system should feel predictable under stress: opening a new root, expanding folders, scrolling dense PDFs, and running searches should not cause behavioral surprises (lost markers, stale badges, blocked navigation) even when background workers are active.

Non-Negotiables

- Never mutate source PDF content.
- Keep bundled `pdf.js` assets isolated under `pdfexplore/vendor/pdfjs/`.
- Sidecar failures must degrade gracefully; malformed JSON must never block browsing.
- Preserve backward compatibility for legacy sidecar payloads where currently supported.

System Invariants

- UI responsiveness invariant:
 - User interaction has higher priority than prefetch and marker scans.
 - Background jobs may be delayed/canceled but UI actions should remain immediate.
- Metadata durability invariant:
 - Highlight and view metadata must survive file switches, refreshes, and restarts.
 - Missing/invalid sidecars must never crash or block browsing.
- Badge continuity invariant:
 - Cache, view, and highlight badges must remain coherent across search updates, tree changes, and document switches.
- Scope correctness invariant:
 - Operations that are scope-dependent (search/copy/highlight actions) must use the same effective-scope semantics as the tree model.
- Search-marker interaction invariant:
 - Right-rail search markers should appear progressively during long scans.
 - Newly visible markers should be clickable immediately.
 - Marker build throughput may be reduced or interrupted to preserve click-to-jump responsiveness.

Key Sidecars

- `.pdfexplore-colors.json` : tree color assignments.
- `.pdfexplore-views.json` : per-document multi-view tab sessions.
- `.pdfexplore-highlighting.json` : persistent in-document text highlight ranges.

Settings Files

- Global/shared settings: `mdexplore.settings.json`
- pdfexplore-specific settings: `pdfexplore.settings.json`

Rule: keep shared constants in `mdexplore.settings.json` ; keep pdfexplore-only runtime settings in `pdfexplore.settings.json` .

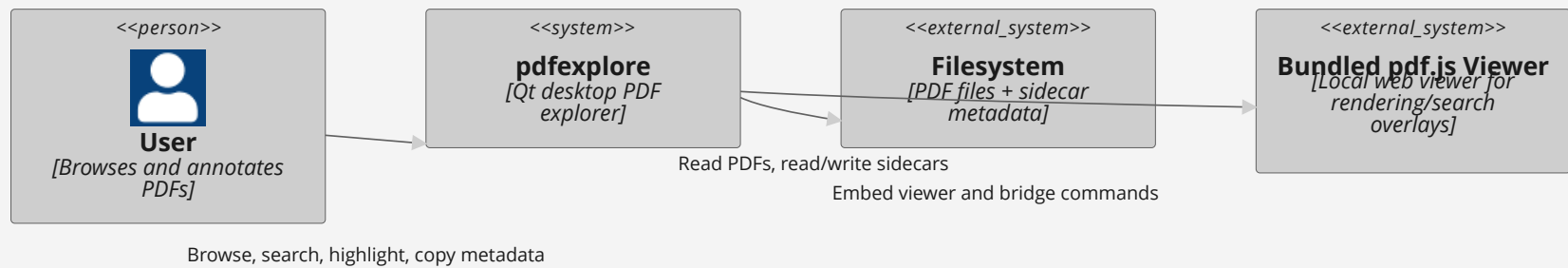
`pdfexplore.settings.json` sections currently include:

- `app` : window/runtime/cache/search/prefetch settings
- `viewer_bridge` : right-rail marker/search behavior tuning
- `tree` : file-tree PDF constants (`.pdfexplore-colors.json` , extension/icon config)

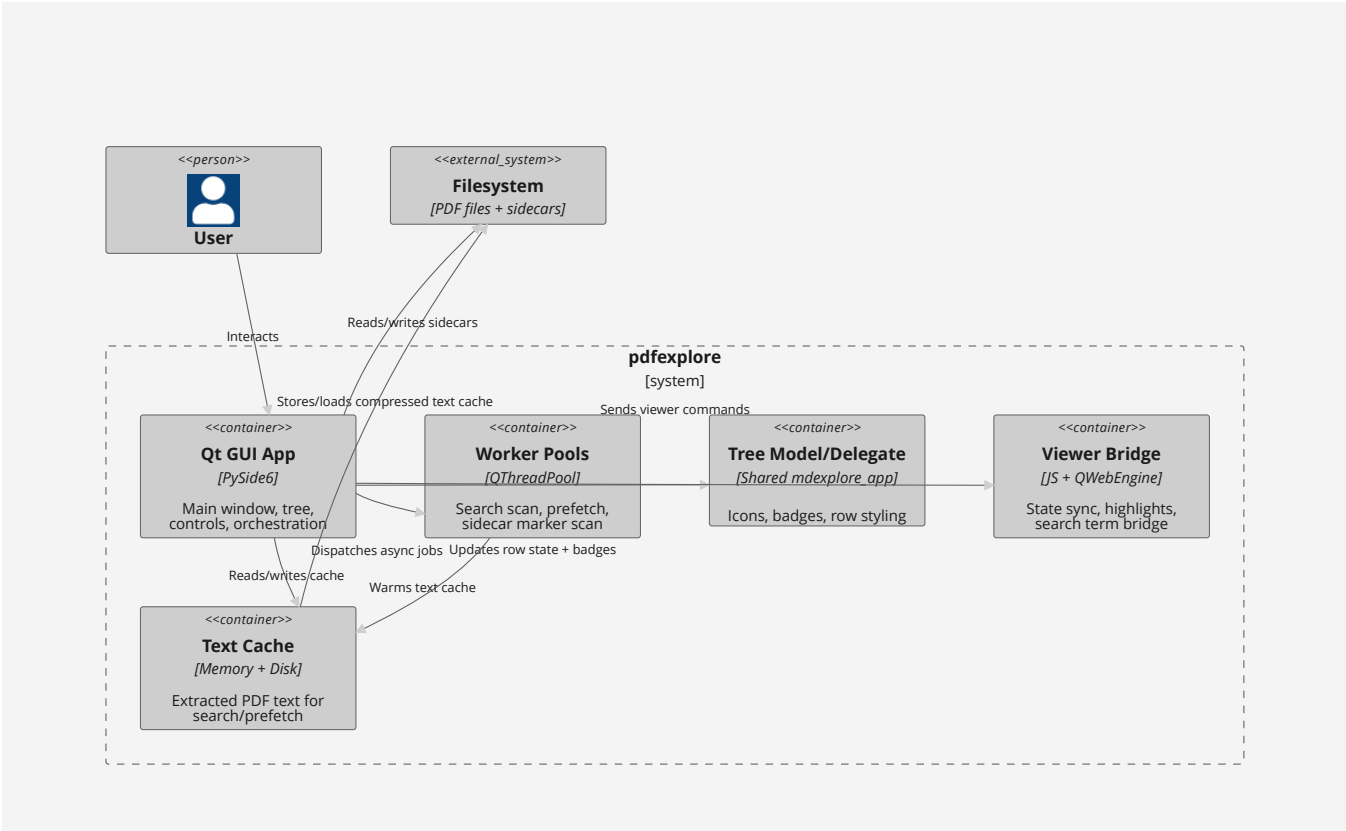
Sidecar Philosophy

- Sidecars store intent/state, not source transformations.
- Writes should avoid noisy churn (do not persist trivial default state unless required).
- Reads should be permissive:
 - tolerate partial payloads,
 - accept legacy shapes when feasible,
 - sanitize malformed entries instead of failing the pipeline.

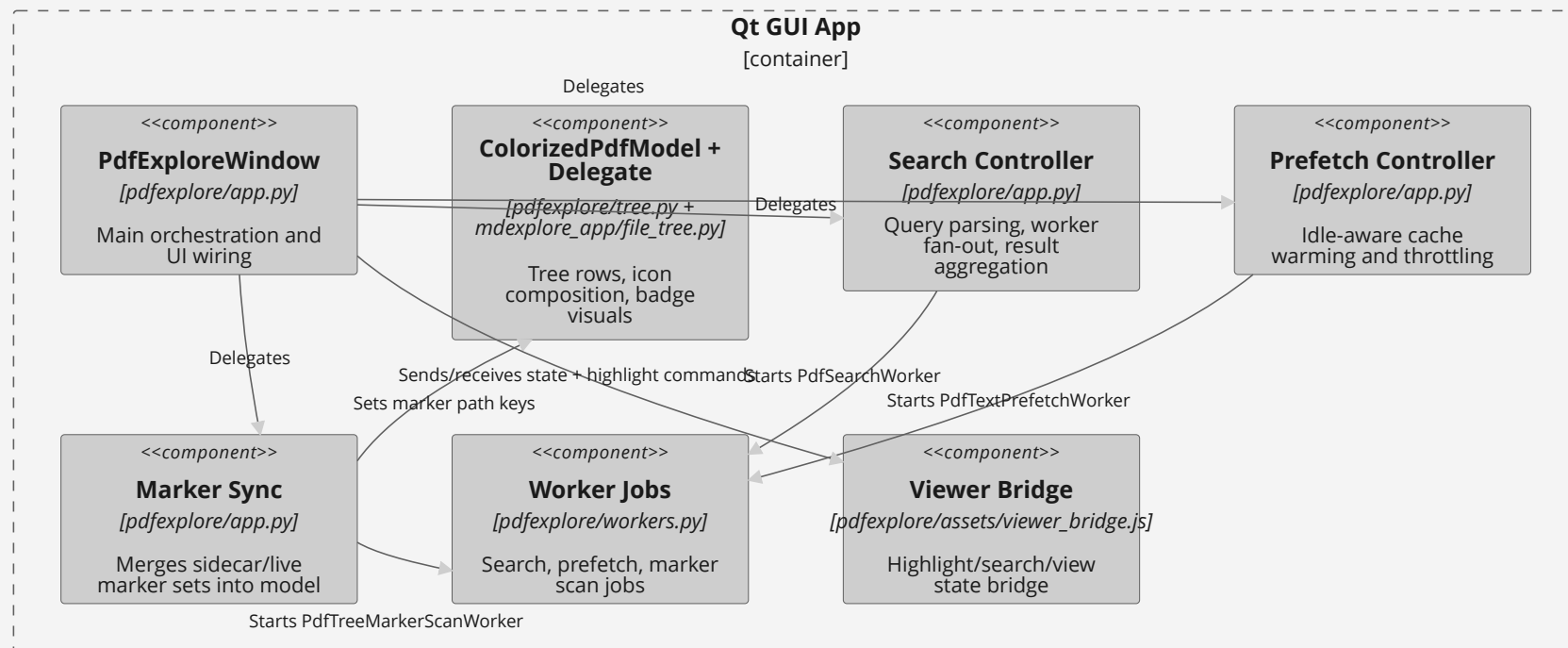
C4 Context



C4 Container



C4 Component (GUI App)



Runtime Lifecycle (Narrative)

1. Boot

- App resolves startup root.
- Tree model/delegate are attached.
- Viewer bridge source is loaded.
- Timers and worker pools are initialized.

2. Root Activation

- Root path is set in the filesystem model.
- Tree state and marker caches are rebuilt.
- Marker sidecar scan is launched in background.

3. Interactive Work

- User navigates tree, opens PDFs, adds highlights, runs searches.
- Search workers fan out by visible-scope candidate chunks.
- Prefetch worker warms text cache opportunistically when idle.

4. Synchronization

- Marker sync merges scan output with live/in-memory state.
- Tree model receives merged badge sets.
- Viewer bridge applies highlight/search overlays.

5. Shutdown

- Active view state is captured.
- Sidecars/config are persisted.
- No source PDF modifications occur.

Worker Coordination Rules

- Search workers:
 - should be cancellation-aware by request id,
 - must never manipulate widgets directly.
- Prefetch workers:
 - run low-priority,
 - should pause/cancel under interaction pressure,
 - should prefer current-document warmup before broader scope.
- Marker scan workers:
 - produce sidecar-derived marker sets,
 - must be merged with live state to avoid transient badge regressions.

Viewer Bridge Marker Rules

- Marker generation in `pdfexplore/assets/viewer_bridge.js` should remain document-key scoped.
- Per-page extracted text should be cached and reused for repeated searches in the same open PDF.
- Marker generation should run in bounded concurrent batches and publish partial results each batch.
- Long builds should periodically yield to the event loop so input/paint are not starved.
- Marker click navigation should be allowed to interrupt active marker builds, then resume automatically.
- Do not block marker click handlers on build completion.

UI Performance Guardrails

- Avoid introducing filesystem `resolve()` loops inside frequently called marker-sync paths.
- Keep repaint pressure low during heavy tree mutation.
- Coalesce badge updates where practical.
- Prefer root-scoped filtering before expensive merging.

Contributor Playbook

When changing any of the following, update both behavior and docs in the same change:

- Search orchestration or query semantics.
- Prefetch scheduling/throttling behavior.
- Sidecar format interpretation.
- Tree badge rendering/priority rules.
- Viewer bridge highlight/search interactions.

Recommended validation sequence:

1. `python -m py_compile` for touched modules.
2. Ensure diagnostics are clean.
3. Manual smoke checks:
 - open root with medium file count,
 - expand/collapse multiple folders,
 - run search and clear search,
 - add/remove highlight, switch files, return,
 - verify cache and marker badges remain coherent.

Failure Modes to Watch

- Marker disappearance after search or tab switch.
- Prefetch starving indefinitely while UI is idle.
- Excessive UI stalls on root change due to marker merge overhead.
- Crash paths around viewer creation or event/filter hooks.
- Sidecar parse errors cascading into badge/model state resets.

Change Boundaries

- Keep generic tree rendering logic in shared `mdexplore_app` when possible.
- Keep PDF-specific orchestration in `pdfexplore/app.py` and `pdfexplore/workers.py`.
- Keep viewer bridge-specific behavior in JS bridge surface, not scattered widget callbacks.

Maintenance Guidance

- Prefer sharing generic behavior through `mdexplore_app` for long-term parity.
- Keep prefetch throttling interaction-first; regressions should bias toward smooth UI.
- Treat marker badge continuity as correctness-critical: highlight and cache badges must survive tab switches, searches, and root navigation.
- Treat marker click latency as correctness-critical in the viewer bridge: visual marker speed without click responsiveness is a regression.
- Preserve current UX contracts:
 - scope-aware operations align with visible tree behavior,
 - copy metadata merge semantics match `mdexplore` expectations,
 - view-tab model remains compatible with existing sidecars.

Documentation Contract

Any change to runtime behavior in these areas must be reflected in `pdfexplore/README.md` :

- Sidecar behavior or compatibility.
- Search/prefetch throttling semantics.
- Marker badge derivation/merge behavior.
- User-visible workflow changes in top controls or preview context menu.